

The Linux File System

Module 8.5

- 8.5.1 The File System Types in Linux
- 8.5.2 Linux Roles and File Permissions
- 8.5.3 Hard Links and Symbolic Links

The File System Types in Linux

- 1 Linux supports multiple file system types: ext4 (default), xfs, btrfs, and tmpfs for RAM-based storage
- 2 ext4 features journaling, support for large files up to 16 TB, and is backward-compatible with ext2/ext3
- 3 XFS is optimized for large files and high-performance I/O; it is the default file system on Red Hat and CentOS
- 4 Btrfs (B-tree FS) supports snapshots, copy-on-write, and built-in RAID, making it ideal for modern workloads
- 5 Use `lsblk -f`, `df -T`, or `blkid` to identify mounted file systems and their types on a running system

Linux Roles and File Permissions

- 1 Every file has three permission sets: owner (user), group, and others — each with read (r), write (w), execute (x) bits
- 2 View permissions with `ls -l`; the mode string (e.g., `-rwxr-xr--`) encodes file type plus three 3-bit permission triplets
- 3 Change permissions with `chmod` using symbolic notation (`chmod u+x file`) or octal notation (`chmod 755 file`)
- 4 Change ownership with `chown user:group file`; change the group only with `chgrp group file`
- 5 Special bits extend the model: `setuid (4xxx)` runs file as owner, `setgid (2xxx)` inherits group, `sticky (1xxx)` restricts deletion

Hard Links and Symbolic Links

- 1 A hard link is a directory entry pointing to the same inode; all hard links share the same data blocks and metadata
- 2 A symbolic (soft) link is a file whose content is a path to another file; it can cross file systems and link to directories
- 3 Create hard links with 'ln source link'; create symbolic links with 'ln -s target link'
- 4 Deleting the original file does not break hard links (inode persists), but it does break symbolic links (dangling symlink)
- 5 ls -li reveals inode numbers — identical for hard links; readlink -f resolves the full canonical path of a symlink